

知序：基于 HarmonyOS 的本地优先学习管理应用

2413575 柯云超

南开大学计算机学院
移动应用开发课程设计答辩

2026 年 6 月

汇报提纲

- 1 项目定位
- 2 技术路线
- 3 前端核心实现
- 4 后端与同步
- 5 演示与总结

一页说清：知序解决什么

背景只抓一个问题：学习资料、任务、专注记录和复盘常分散在不同工具，数据难以形成连续闭环。

项目目标：在 HarmonyOS 端侧完成“资料整理 – 任务分解 – 番茄执行 – 复盘反馈”，后端同步和远程 AI 作为增强能力。

- 离线状态下，任务、资料、番茄和复盘仍可运行。
- 网络可用时，支持 AI 提炼、图片提取任务和云同步。
- 答辩重点展示移动端、端侧数据库、原生能力和后端接口的完整链路。

ArkTS

前端实现

ArkUI + Navigation + 原生能力

RDB

本地优先

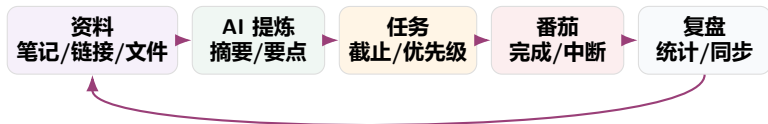
业务数据首先写入端侧数据库

Spring Boot

后端服务

REST + MyBatis + JWT +
MySQL/H2

业务闭环：从资料到复盘



反馈下一轮学习安排



今日工作台

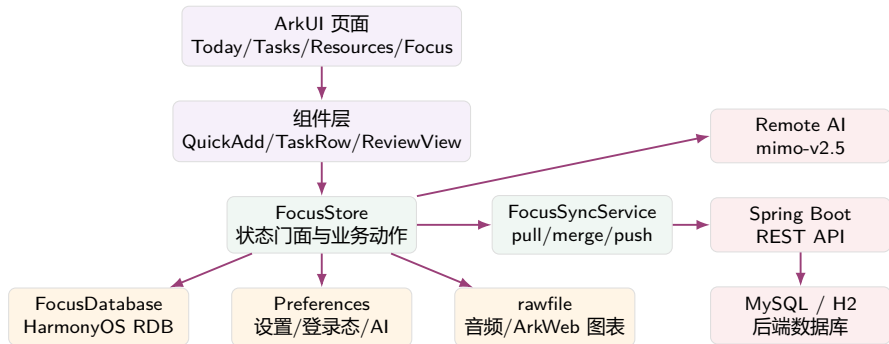


任务管理



复盘统计

总体技术路线：端侧闭环，服务端增强



核心原则

页面只负责展示和交互，数据读写、AI 请求、同步、音频播放都下沉到服务层；本地 RDB 是主数据源，远程服务只增强体验和备份能力。

技术选型与代码落点

技术点	代码位置	作用
ArkUI / Navigation RDB	Index.ets components/* FocusDatabase	底部 Tabs、二级页面栈、组件化界面。 建表、迁移、CRUD、dirty 标记和软删除。
Store 门面	FocusStore	汇总项目、任务、资料、番茄并刷新统计快照。
AI / OCR	Index AiCoachService	端侧 OCR、文本解析、视觉兜底和空响应诊断。
ArkWeb / TaskPool AVPlayer	ReviewView FocusStore AmbientService	本地图表展示与复盘统计后台预计算。 循环播放 rawfile 白噪音并释放资源。
Spring Boot	focus-server	JWT 登录、业务接口、同步接口和 MyBatis 数据访问。

前端分层：Index 编排，Store 接业务

页面层

Index.ets 维护页面状态、路由栈、Tab 切换和用户事件回调。

组件层

TodayView、TasksView、ResourcesView、FocusSessionView、ReviewView 只接收状态和回调。

服务层

FocusStore 调用数据库、设置、AI、同步和音频服务，完成后刷新 UI 状态。

一次任务新增的链路

- 1 QuickAddTaskCard 收集标题、备注、截止时间、项目和象限。
- 2 Index.ets 组装 QuickTaskDraft。
- 3 FocusStore.addTaskFromDraft 规范化字段。
- 4 FocusDatabase.addTask 写入 RDB。
- 5 FocusStore.refresh 重新读取并触发页面刷新。

核心功能一：快速添加与图片提取任务



任务页：多行输入、截止时间、图片
候选确认

功能实现

- 手动输入：第一行作为标题，后续行可转为备注或子任务。
- 时间输入：支持“今天 18:00 / 明天 09:00 / 6 小时后”等自然表达。
- 图片输入：拍照或上传图片，CoreVision OCR 优先识别文本；文本解析失败时，再把图片交给配置的 mimo-v2.5 做视觉兜底。
- 安全写入：AI 结果先作为候选草稿展示，用户确认单条或全部后才写入 RDB。

OCR 优先

视觉兜底

候选确认

RDB 落库

核心功能二：资料库到行动任务



资料页：录入、文件引用、AI 提炼

资料处理链路

- 1 资料可来自笔记、网页链接、PDF 摘录或本地文件引用。
- 2 `study_resources` 保存标题、类型、正文摘录、摘要和复习时间。
- 3 远程 AI 可生成摘要、要点和下一步任务建议。
- 4 `FocusStore.createTaskFromResource` 把资料提炼结果转成可执行任务。

设计边界

当前保存文件引用和用户备注，不直接解析完整 PDF/DOCX/PPTX，保证课程项目范围内实现稳定。

核心功能三：番茄专注与复盘统计



番茄专注



复盘统计



我的入口

专注实现

用开始时间戳计算剩余时间，减少切后台后纯倒计时漂移；完成或中断后写入 pomodoros 表，并通过 AVPlayer 播放本地白噪音。

复盘实现

复盘统计由 FocusStore 聚合任务和番茄记录，TaskPool 预计算统计快照，ArkWeb 加载本地 charts/index.html 展示图表。

端侧数据模型：本地优先的基础

表	关键字段	说明
projects	name、updated_at、is_deleted、dirty	课程/学习项目；参与软删除和同步。
tasks	title、due_at、status、request_id、tags	任务主体；支持象限、截止时间、标签、子任务和幂等同步。
pomodoros	task_id、start_at、duration、completed	每次专注的事实记录，支撑统计和任务番茄数。
resources	title、source_type、summary、review_due	资料内容、AI 提炼结果和复习线索。
app_meta	key、value	保存演示数据刷新版本等轻量元信息。

同步相关字段

dirty 控制本地待上推记录；is_deleted 表示删除墓碑；client_request_id 降低重复提交造成的重复任务或重复番茄。

服务端技术路线：REST + MyBatis + JWT

后端分层

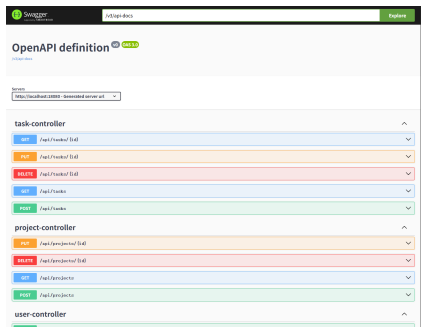
- Controller：用户、项目、任务、番茄、统计、同步接口。
- DTO/Entity：请求响应对象与数据库实体。
- Mapper/XML：MyBatis SQL 映射，支持分页、统计和增量查询。
- Config/Service：JWT 拦截器、跨域配置和令牌签发。

Spring Boot

MyBatis

JWT

MySQL/H2



Swagger/OpenAPI 接口文档

后端接口覆盖

模块	接口	作用
用户	/api/user/login、 /register	登录、注册、连通性检查，返回 JWT。
项目	/api/projects	课程项目增删改查和软删除。
任务	/api/tasks	任务分页、创建、更新、删除。
番茄	/api/pomodoros	创建专注记录、按范围查询。
统计	/api/stats/summary	后端统计汇总。
同步	/api/sync/pull、/push	增量拉取和本地变更上推。

鉴权方式

登录成功后移动端在 `Authorization: Bearer <token>` 中携带 JWT，后端拦截器校验后将用户标识写入请求上下文，业务接口不信任客户端传入的用户编号。

双向同步：先拉取，再上推



云同步页：地址、登录和同步操作

客户端流程

- 1 读取 `lastServerTime` 作为增量游标。
- 2 调用 `pullFromServer` 拉取服务端变更。
- 3 按 `updatedAt` 合并到本地 RDB。
- 4 查询本地 `dirty=1` 数据并调用 `pushToServer`。
- 5 上推成功后清除 `dirty`，保存新的服务端时间。

serverTime 游标

dirty 增量

updatedAt LWW

同步一致性：避免重复和“删除后恢复”



- **重复提交**：任务和番茄记录带 `clientRequestId`，服务端先查同一请求标识再决定插入或更新。
- **冲突处理**：客户端和服务端都比较 `updatedAt`，保留不旧于已有版本的数据。
- **删除传播**：项目和任务保留 `isDeleted tombstone`，同步时继续上推删除语义。

演示路径：按技术链路讲



1. 今日任务聚合



2. 快速添加/图片候选



3. 后端同步

建议答辩演示顺序

启动 H2 后端 → 登录 demo/secret → 在任务页快速添加或上传截图 → 开始番茄专注 → 查看复盘统计 → 执行同步并展示 Swagger 接口。

HarmonyOS 前端

- ArkTS / ArkUI 声明式页面与组件拆分。
- Navigation / NavPathStack 二级页面栈。
- RDB、Preferences、AVPlayer、ArkWeb、TaskPool、FormAbility。

后端与工程化

- Spring Boot REST API 与统一响应封装。
- MyBatis XML SQL、MySQL/H2 双数据库。
- JWT 鉴权、增量同步、构建脚本和演示数据。

已完成

- 端侧本地优先学习闭环。
- 任务、资料、专注、复盘、云同步完整链路。
- 前端原生能力、后端接口和数据库同步均可演示。

后续改进

- 更细粒度的同步冲突合并和失败重试。
- PDF/DOCX/PPTX 正文解析与资料问答。
- 平板布局、无障碍和真机后台计时可靠性。

谢谢！
欢迎老师批评指正